

Shared Memory

IPC: Shared Memory

Learning Objectives

- Be able to describe when and why pipes and sockets become performance bottleneck
- Be able to implement shared memory communication between parent and child processes using POSIX `shm_open()` and `mmap()`.
- Be able to evaluate which IPC mechanism (signals, pipes, sockets, or shared memory) is most appropriate for different scenarios.

Complementary Reading: *Beej's Guide to Interprocess Communication* — Section on Share Memory (free online).

Pipe/Socket limitations

- Each read/write to a pipe/socket needs to cross into the kernel
 - Expensive context switches
 - Expensive data copies.
- What if we try to share a large amount of data?
- What if we want to share that data across multiple child processes?
- What if multiple processes could access *the same memory region* directly?

Aside: Separating the Control Plane and Data Plane

- **Control Plane**
 - Mechanisms that manage *who* can do something or how to use resources.
 - Examples: `open()`, `socket()`, `pipe()`, `mmap()`, `shm_open()`.
- **Data Plane:**
 - The actual *work* of the thing.
 - Examples: reading/writing data after setup